

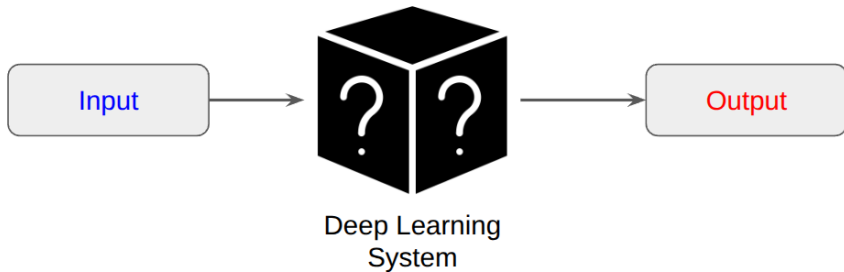
DEEP LEARNING

Fundamentals of Deep Learning: Part I

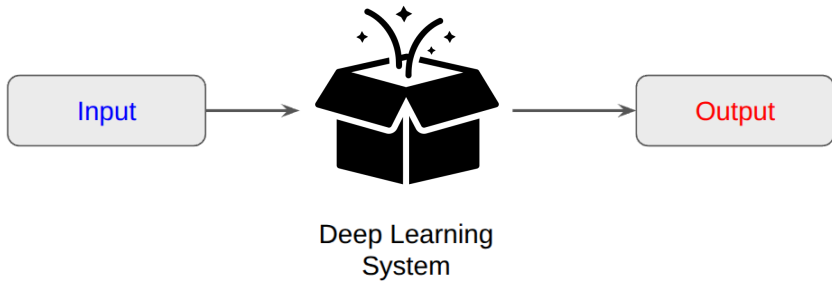
Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Recap

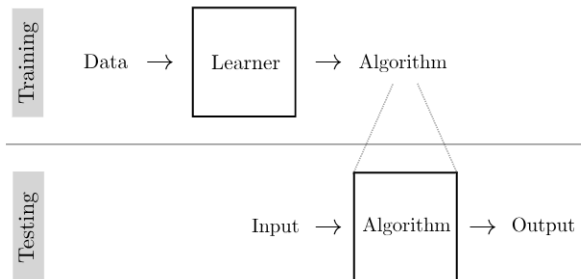


Today

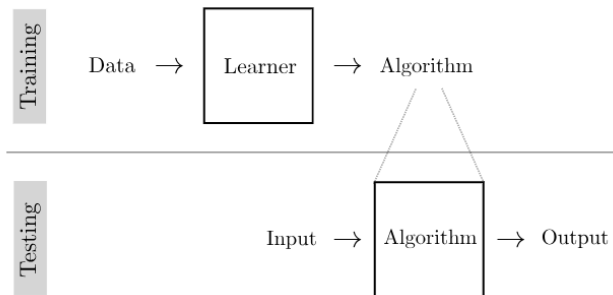


What is "Learning" in Deep Learning?

- The goal of **learning** is to extract lessons from past experiences (or examples or data) in order to solve future problems.
- What does learning entail?
 - ▶ It involves searching for an "**algorithm**" that solves past instances of the problem.
 - ▶ Once we have learned the algorithm, we can use it to future instances of the problem.

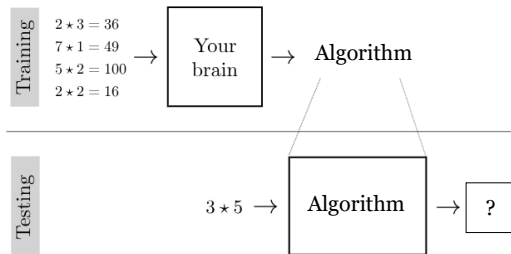


What is "Learning" in Deep Learning?



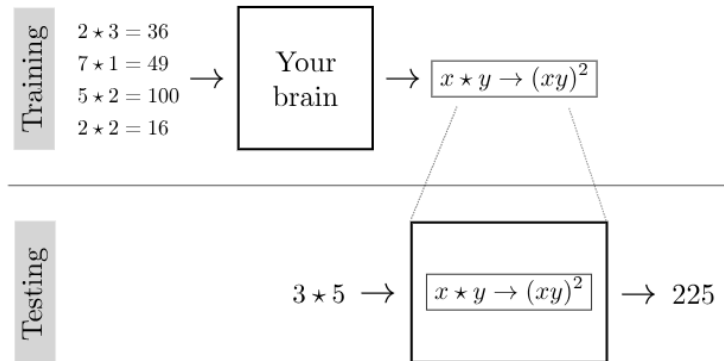
- Learning consists of two phases:
 - ▶ The **training** phase: where we search for an algorithm using past examples of the problem (also called as training data)
 - ▶ The **testing** phase: where we deploy our learned algorithm to solve new instances of the problem (also called as test data)

Learning from Examples: A Motivating Example



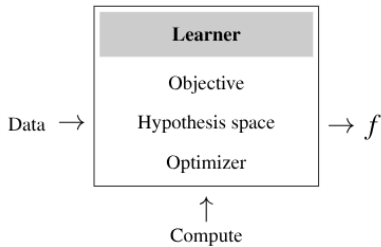
- Consider that you do not know what the operator $\star$ means. You are only given a set of examples for finding the meaning of the operator $\star$:
 1. {input: [2, 3], output: 36}
 2. {input: [7, 1], output: 49}
 3. {input: [5, 2], output: 100}
 4. {input: [2, 2], output: 16}
- Q: Find the algorithm and then apply your algorithm on the test data $3 \star 5 = ?$

Learning from Examples: A Motivating Example (Solution)



- The algorithm that we found is: $x \star y = (xy)^2$.
- More generally speaking, given some data we want to come up with an algorithm that will do well on future data (or in the testing phase).

Three Key Ingredients in Learning



- **Objective:** What does it mean for the learner to succeed, or, at least, to perform well?
 - **Hypothesis space:** What is the set of possible mappings from inputs to outputs that we will search over?
 - **Optimizer:** How, exactly, do we search the hypothesis space for a specific mapping that satisfies the objective.
- The Learner outputs an algorithm or a function f that maps inputs $\mathbf{x} \in \mathcal{X}$ to outputs $\mathbf{y} \in \mathcal{Y}$.
 - We denote this mathematically as: $f : \mathcal{X} \rightarrow \mathcal{Y}$, where f is also called the **mapping function**.

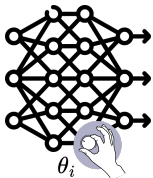
Learning through Parameterization

- A natural question to ask: what exactly in the function f are we trying to learn?
- We will "parameterize" the function f with parameters Θ and will be learning those parameters.
- We can write the parameterized function as:

$$f_{\Theta} : \mathcal{X} \rightarrow \mathcal{Y},$$

where $\Theta = [\theta_1, \theta_2, \dots, \theta_n]$ is a vector with n variables. We also call Θ as **weights**.

- "Tuning" the values of all the elements in Θ is called learning the f .



$Y = \text{"dog"}$

$Y = \text{"cat"}$

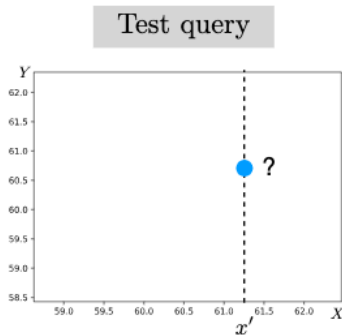
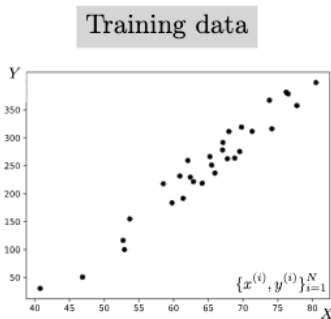
$Y = \text{"fish"}$

Learning: A Case Study

- One of the simplest learning problems is known as **linear least-squares regression**. In this setting, we aim to model the relationship between two variables, x and y , with a line.
- Let's say x represents the temperature outside, and y represents the number of people at the beach.
- **Goal:** Find (or learn) the relationship between outside temperature and the number of people at the beach.
- **Training data:** We have training data (or examples) of the format $\{\text{temperature outside, number of people at the beach}\}$. This is denoted as $\{x^{(i)}, y^{(i)}\}_{i=1}^N$. We often call y the **ground-truth**.
- Once we have learned the relationship between x and y , we are given a new test sample (or test query) x' , and we need to predict the value of y .

Learning: A Case Study

- Visually, the training data and the test query is given as:



Learning: A Case Study

- We will first start with the ingredient in learning: [hypothesis space](#).
- We hypothesize that x and y follow a linear relationship. In other words, the hypothesis space is linear functions.
- The prediction of the model (or parameterized function) is given as:

$$\hat{y} = f_{\Theta}(x) = \theta_1 x + \theta_0,$$

where $\Theta = [\theta_0, \theta_1]$ are the two parameters that are unknown and are being learned.

Learning: A Case Study

- The second ingredient in learning is **objective**, which measures how good the model is at making predictions.
- For example, if for $x = 59$, the ground-truth $y = 34$, and the model predicted $\hat{y} = 33$, then we can reasonably say that the model succeeded.
- On the other hand, if the model predicted $\hat{y} = 10$, then we can reasonably say that the model has failed, because the prediction is way off from the ground-truth $y = 34$.
- More generally, we quantify success for the i -th training data point using a **loss function** or **objective function**:

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = (\hat{y}^{(i)} - y^{(i)})^2$$

- The total objective for all the training examples is given as:

$$J(\Theta) = \sum_{i=1}^N \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = \sum_{i=1}^N \mathcal{L}(f_{\Theta}(x^{(i)}), y^{(i)})$$

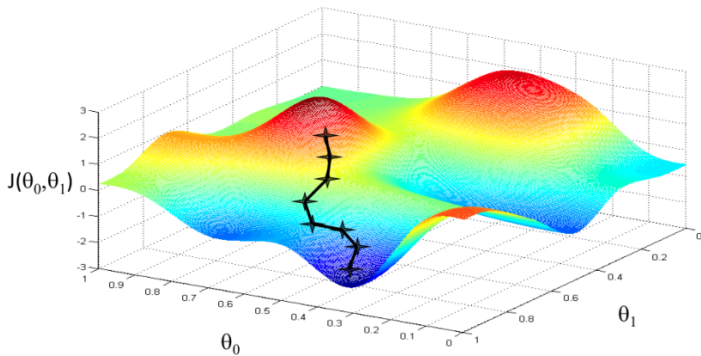
Learning: A Case Study

- Our goal is to minimize the objective/loss:

$$J(\Theta) = \sum_{i=1}^N (f_{\Theta}(x^{(i)}) - y^{(i)})^2 = \sum_{i=1}^N (\theta_1 x^{(i)} + \theta_0 - y^{(i)})^2$$

- We are now ready for the final ingredient – the **optimizer** – whose goal is to find the optimal values of the parameters $\Theta = [\theta_0, \theta_1]$.
- **Naive approach:** try a bunch of random values for Θ until we find the values that give us maximum success.
- The naive approach is slow since we are searching for good solutions blindly.
- We need something smarter!
- A more efficient optimization algorithm to find the optimal parameters Θ is through **gradient descent**.

Gradient Descent Optimization

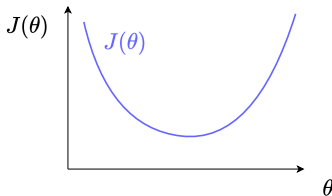


Gradient Descent

- To recap, our goal is to **minimize** the objective function.
- Let's simplify our model by assuming that it has only one learnable parameter θ :

$$J(\theta) = \sum_{i=1}^N (\theta x^{(i)} - y^{(i)})^2$$

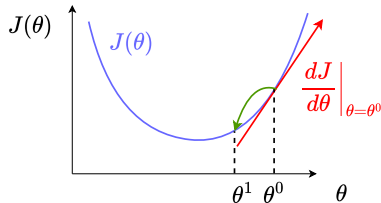
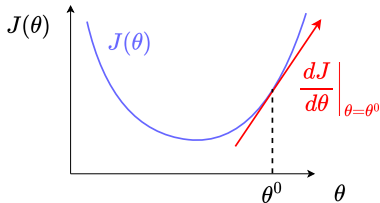
- If we plot the objective (or loss), it would look something like below. We also call it the **loss landscape**.



- Our goal is to find the value of θ where $J(\theta)$ is minimum (also called as the **minima**), and we want to find an approach that is better than random guessing.

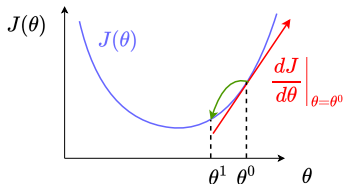
Gradient Descent

- Gradient descent is an **iterative algorithm** that will help us find the minimum.
- Intuitively, it works as follows:
 1. Make an initial guess of θ , randomly. Let's say we chose a value θ^0 .
 2. Compute the derivative of the objective with respect to θ , ie, $\frac{dJ(\theta)}{d\theta}$. This derivative tells us: in which direction $J(\theta)$ is increasing the fastest at the given point θ^0 .
 3. Since our goal is to minimize the objective, we will move in the direction **opposite** to the derivative.
 4. So we will shift our estimate of θ from θ^0 to θ^1 .
 5. Repeat steps 2, 3 and 4 until we are the bottom of the curve.

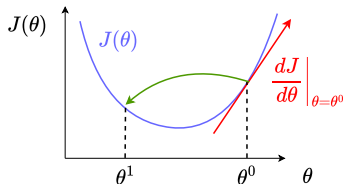


Gradient Descent

- Next question arises, how much should we shift our estimate of θ ?



Option A



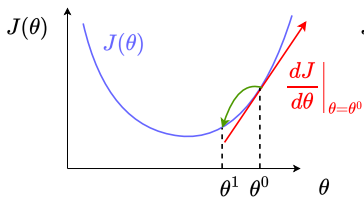
Option B

- Question:** Should we choose Option A or Option B?
- The amount by which we will shift our estimate of θ depends on a α , which is called the **learning rate**. α controls how big a step you take on each update.
- The weight (or parameter) **update rule** is given as:

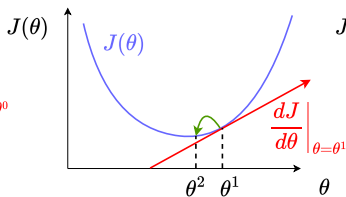
$$\theta \leftarrow \theta - \alpha \frac{dJ(\theta)}{d\theta}$$

Gradient Descent

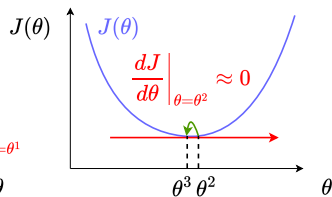
- Gradient descent is an **iterative algorithm**. It keeps updating the weights until one of the following criteria are met:
 - the weights do not change anymore (or the derivative becomes close to zero)
 - the weights have been updated for a pre-defined number of iterations
 - other criteria exist, which will be discussed in a future lecture



Step 1



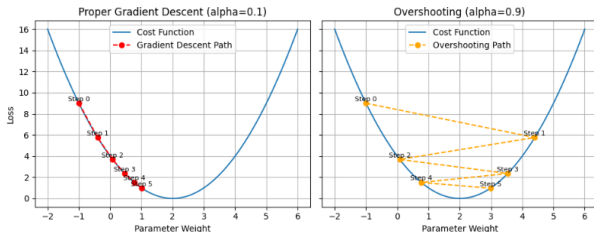
Step 2



Step 3

Gradient Descent

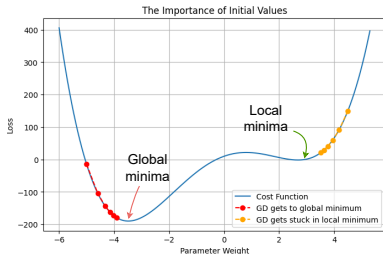
- The learning rate α controls how well you can learn and should be set carefully.



- ▶ If α set to too small, then it will take many steps to reach the minima of the objective function.
- ▶ If α set to too high value (eg, $\alpha = 0.9$) then **overshooting** can occur, like a ball bouncing between two walls.
- ▶ Only if set carefully (eg, $\alpha = 0.1$), then it will converge to the minima reasonably fast.
- Therefore, α is called a **hyperparameter** because typically we don't learn it, but set it ourselves (more on how to set it later).

Gradient Descent

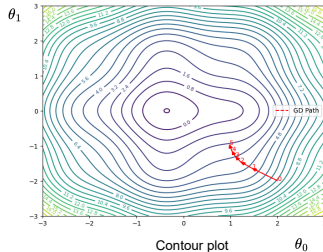
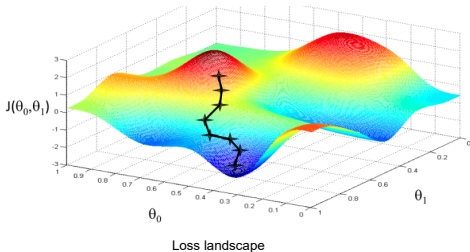
- The loss landscape is not always benign (or mathematically called **convex**).
- Sometimes, we can have non-convex loss landscapes, such as the one below. The place where you end up depends on where you start or the initial value θ^0 .



- ▶ A **global minima** is a point in the loss landscape where the value of the loss/objective is the lowest globally.
- ▶ A **local minima** is a point in the loss landscape where the value of the loss/objective is the lowest, in a local neighbourhood.
- We always desire to reach the global minima but that is always not feasible.

Gradient Descent

- Extending the idea to more than two parameters $\Theta = [\theta_0, \theta_1]$, the loss landscape will look as below.



- In that case we need to compute **partial derivatives** instead of derivatives. The update rule for each parameter is given as:

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

Gradient Descent

- So far we have been trying to learn the relationship between an input x (a scalar value) and an output y (a scalar value), using the model:

$$\hat{y} = f_{\Theta}(x) = \theta_1 x + \theta_0,$$

- The input can also be a vector $\mathbf{x} = [x_1, x_2, \dots, x_d]$ of d elements.
- For example, instead of just temperature outside, we can also have more than one input variable, such as {temperature outside, wind speed, weekend or not, month of the year, ...}
- In that case the model (still linear) is written as:

$$\hat{y} = f_{\Theta}(\mathbf{x}) = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d + \theta_0,$$

- The goal is to find the optimal values of the $d + 1$ parameter values:
 $\Theta = [\theta_1, \theta_2, \dots, \theta_d, \theta_0]$.

Gradient Descent

- Further extending from two parameters to $d + 1$ parameters (ie, $\Theta \in \mathbb{R}^{d+1}$), we combine all the partial derivatives in a gradient vector, we get the following:

$$\nabla_{\Theta} J = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \frac{\partial J}{\partial \theta_2} \\ \vdots \\ \frac{\partial J}{\partial \theta_d} \\ \frac{\partial J}{\partial \theta_0} \end{bmatrix} \in \mathbb{R}^{d+1}$$

- The weight update rule is given as:

$$\Theta \leftarrow \Theta - \alpha \nabla_{\Theta} J$$

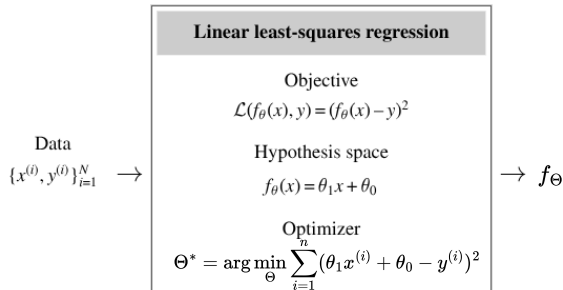
- The optimization algorithm gradient descent gets its name from the word gradient.

Learning: Back to the Case Study

- The output of the optimization algorithm (ie, gradient descent) is the optimal set of parameters or weights Θ^* , written as:

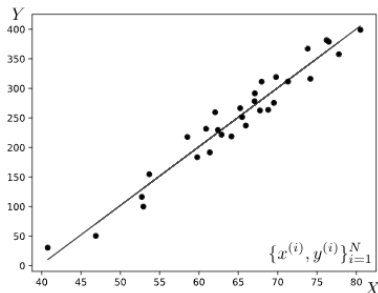
$$\Theta^* = \arg \min_{\Theta} \sum_{i=1}^n (\theta_1 x^{(i)} + \theta_0 - y^{(i)})^2$$

- The whole pipeline can be summarized as:

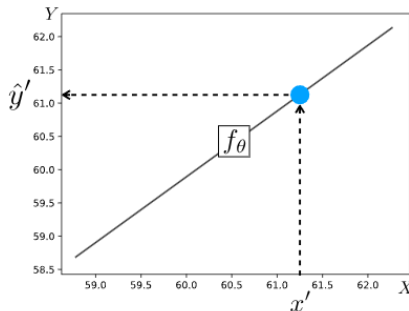


Learning: A Case Study

Training data

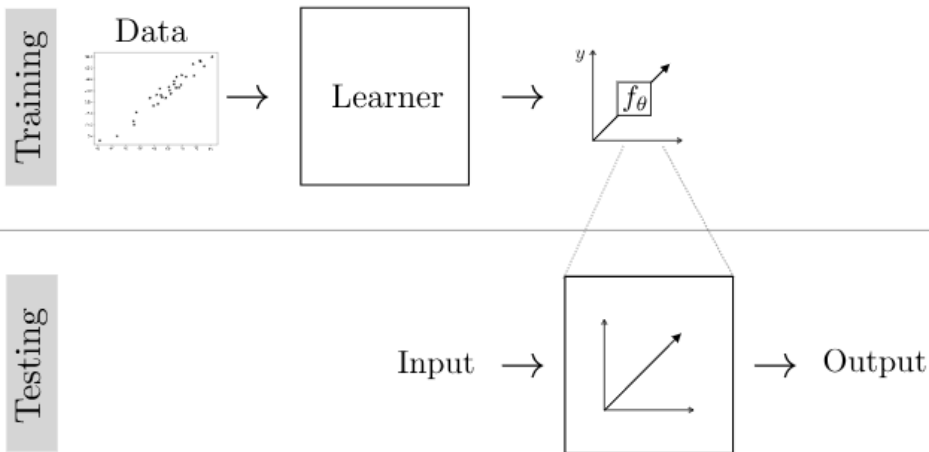


Test query

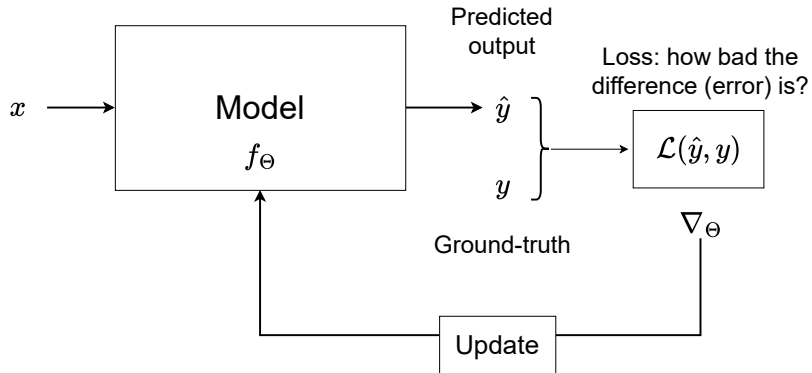


- Since we chose the hypothesis space to be a linear function, the learned function is a straight line.
- Given a test query x' , the learned model f_θ makes a prediction $\hat{y}'$.

Learning: Summary



Modular Learning System



Extra Readings

- Dive into Deep Learning (<https://d2l.ai/index.html>). Chapter 3 (3.1 - 3.5)

Credits and Acknowledgements

- Some slides have been adapted from Torralba, Isola and Freeman